

A Level Computer Science (9618)

Topical Past Paper Worksheet

Topic 4: Processor Fundamentals

Subtopic 4.3: Bit Manipulation

Concept 4.3.1: Binary Shifts (Logical, Arithmetic, Cyclic)

Total Questions: 11

Mark Schemes begin on Page 13

Question 1 | Source: 9618_w25_qp_11 (Q4.a)

- 4 The table shows part of the instruction set for a processor. The processor has one register, the Accumulator (ACC).

Instruction		Explanation
Opcode	Operand	
AND	#n / Bn / &n	Bitwise AND operation of the contents of the ACC with the operand
AND	<address>	Bitwise AND operation of the contents of the ACC with the contents of <address>
XOR	#n / Bn / &n	Bitwise XOR operation of the contents of the ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of the ACC with the contents of <address>
OR	#n / Bn / &n	Bitwise OR operation of the contents of the ACC with the operand
OR	<address>	Bitwise OR operation of the contents of the ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end.
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end.
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #127 B denotes a binary number, e.g. B10010001 & denotes a hexadecimal number, e.g. &4A		

- (a) The ACC currently contains the following positive binary integer:

0	0	0	1	1	1	1	0
---	---	---	---	---	---	---	---

Write a bit manipulation instruction that uses a binary shift to change the contents of the ACC to:

0	1	1	1	1	0	0	0
---	---	---	---	---	---	---	---

Instruction [1]

Question 3 | Source: 9618_s25_qp_11 (Q8.b.iii)

- (b) The table shows part of the instruction set for a processor. The processor has one register: the Accumulator (ACC).

Instruction		Explanation
Opcode	Operand	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(iii) The ACC currently contains the following binary value.

0	1	1	0	1	0	1	1
---	---	---	---	---	---	---	---

Write the result after the instruction LSR #5 is run.

--	--	--	--	--	--	--	--

[1]

Question 4 | Source: 9618_w24_qp_13 (Q8.c)

(c) Describe the difference between a right logical binary shift and a right arithmetic binary shift.

.....

.....

.....

..... [2]

Question 5 | Source: 9618_w23_qp_11 (Q8.c.ii)

(c) The following table shows another part of the instruction set for the same processor.

Instruction		Explanation
Opcode	Operand	
AND	Bn	Bitwise AND operation of the contents of ACC with the operand
XOR	Bn	Bitwise XOR operation of the contents of ACC with the operand
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left hand end
# denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(ii) The current contents of the ACC are:

0	0	0	1	0	1	1	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

LSR #3

.....

.....

--	--	--	--	--	--	--	--

[1]

Question 6 | Source: 9618_s23_qp_11 (Q3.d.vi)

3 A computer has an Operating System (OS).

(d) The computer stores data in binary form.

(vi) Show the result of a 3-place right logical shift on the binary number:

11001100

..... [1]

Question 7 | Source: 9618_s23_qp_12 (Q4.d)

4 Data in a computer is stored in binary form.

(d) Show the result of a 2-place left logical shift on the binary number:

01001111

..... [1]

Question 8 | Source: 9618_w22_qp_11 (Q6.b.iv)

- 6 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	#n	Compare the contents of ACC with number n
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(b) The following table shows another part of the instruction set for the processor.

Instruction		Explanation
Opcode	Operand	
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	Bn	Bitwise AND operation of the contents of ACC with the binary number n
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	Bn	Bitwise XOR operation of the contents of ACC with the binary number n
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	Bn	Bitwise OR operation of the contents of ACC with the binary number n
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(iv) The current contents of the ACC are:

1	0	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Show the result after the execution of the following instruction.

LSR #2

.....

.....

--	--	--	--	--	--	--	--

[1]

Question 9 | Source: 9618_w22_qp_12 (Q7.b.iii)

- 7 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store the contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	<address>	Compare the contents of ACC with the contents of <address>
CMI	<address>	Indirect addressing. The address to be used is at the given address. Compare the contents of ACC with the contents of this second address
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
END		Return control to the operating system
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(b) The following table shows another part of the instruction set for the processor.

Instruction		Explanation
Opcode	Operand	
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	Bn	Bitwise XOR operation of the contents of ACC with the binary number n
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

The contents of memory addresses 50 and 51 are shown:

Memory address	Data value
50	01001101
51	10001111

(iii) The current contents of the ACC are:

0	1	0	1	0	0	1	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

LSL #3

.....

.....

--	--	--	--	--	--	--	--

[1]

Question 10 | Source: 9618_w22_qp_13 (Q6.b.iii)

- 6 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Opcode	Operand	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
MOV	<register>	Moves the contents of the accumulator to the given register (IX)
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
SUB	#n	Subtract the denary number n from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	#n	Compare the contents of ACC with number n
CMI	<address>	Indirect addressing. The address to be used is at the given address. Compare the contents of ACC with the contents of this second address
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(b) The following table shows another part of the instruction set for the processor.

Instruction		Explanation
Opcode	Operand	
AND	#n	Bitwise AND operation of the contents of ACC with the operand
AND	Bn	Bitwise AND operation of the contents of ACC with the binary number n
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n	Bitwise OR operation of the contents of ACC with the operand
OR	Bn	Bitwise OR operation of the contents of ACC with the binary number n
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001101		

(iii) The current contents of the ACC are:

1	0	0	1	1	1	0	1
---	---	---	---	---	---	---	---

Show the contents of the ACC after the execution of the following instruction.

LSR #2

.....

.....

--	--	--	--	--	--	--	--

[1]

Question 11 | Source: 9618_s21_qp_11 (Q3.c.i)

- 3 A processor has one general purpose register, the Accumulator (ACC), and several special purpose registers.

(c) (i) The Accumulator currently contains the binary number:

0	0	1	1	0	1	0	1
---	---	---	---	---	---	---	---

Write the contents of the Accumulator after the processor has executed the following instruction:

LSL #2

--	--	--	--	--	--	--	--

[1]

Question 12 | Source: 9618_s21_qp_11 (Q3.c.ii)

- 3 A processor has one general purpose register, the Accumulator (ACC), and several special purpose registers.

(ii) The Accumulator currently contains the binary number:

0	0	1	1	0	1	0	1
---	---	---	---	---	---	---	---

Identify the mathematical operation that the following instruction will perform on the contents of the accumulator.

LSR #3

.....
..... [1]

MARK SCHEMES

Mark Scheme for Question 1 | Source: 9618_w25_ms_11 (Q4.a)

4(a)	1 mark for: LSL #2	1
------	------------------------------	----------

Mark Scheme for Question 2 | Source: 9618_w25_ms_13 (Q2.c)

2(c)	1 mark 1111 0010	1
------	--------------------------------	----------

Mark Scheme for Question 3 | Source: 9618_s25_ms_11 (Q8.b.iii)

8(b)(iii)	1 mark for: 0000 0011	1
-----------	-------------------------------------	----------

Mark Scheme for Question 4 | Source: 9618_w24_ms_13 (Q8.c)

8(c)	1 mark each: • A logical shift moves all bits to the right and inserts zeros in the appropriate leftmost bits • An arithmetic shift moves all bits to the right but copies the sign bit into the Most Significant Bit (MSB)	2
------	--	----------

Mark Scheme for Question 5 | Source: 9618_w23_ms_11 (Q8.c.ii)

8(c)(ii)	0000 0010	1
----------	-----------	----------

Mark Scheme for Question 6 | Source: 9618_s23_ms_11 (Q3.d.vi)

3(d)(vi)	00011001	1
----------	----------	----------

Mark Scheme for Question 7 | Source: 9618_s23_ms_12 (Q4.d)

4(d)	0011 1100	1
------	-----------	----------

Mark Scheme for Question 8 | Source: 9618_w22_ms_11 (Q6.b.iv)

6(b)(iv)	0010 0100	1
----------	-----------	----------

Mark Scheme for Question 9 | Source: 9618_w22_ms_12 (Q7.b.iii)

7(b)(iii)	1001 1000	1
-----------	-----------	----------

Mark Scheme for Question 10 | Source: 9618_w22_ms_13 (Q6.b.iii)

6(b)(iii)	0010 0111	1
-----------	-----------	----------

Mark Scheme for Question 11 | Source: 9618_s21_ms_11 (Q3.c.i)

3(c)(i)									1
	1	1	0	1	0	1	0	0	

Mark Scheme for Question 12 | Source: 9618_s21_ms_11 (Q3.c.ii)

3(c)(ii)	1 mark for correct answer The number is divided by 8 (and only whole number retained)	1
----------	---	----------